

1. Reasoning

Our team analyzed the LibTracker System to find ways to improve its design. We worked together and realized that multiple controllers need to connect to one single database. To manage this system efficiently and save resources, we decided to apply a design pattern to our database connection.

2. Pattern Selection

Selected Pattern:

Singleton Design Pattern

Problem-Solution Fit:

In our Class Diagram, the "System Database" is used by different parts of the application, such as the AuthController and FeedbackController. If the system creates a new database object for every single request, it will waste memory and cause data synchronization problems. The Singleton pattern solves this by ensuring that only one instance of the System Database exists during the whole application lifecycle.

3. Scenario and Implementation

Scenario:

When a Student submits a feedback form, the FeedbackController needs to save this data. Instead of opening a brand new database connection, the controller calls the Singleton class. The class checks if a connection already exists. If yes, it uses the existing one; if no, it creates one.

4. Documentation and Explanation

How it works:

We used Python's `__new__` method in the `SystemDatabase` class. This method controls the object creation. It guarantees that the system cannot create a second database object.

By using this pattern, our system avoids memory leaks and keeps all database operations fully synchronized across different users.

```
1 class SystemDatabase:
2     _instance = None
3
4     def __new__(cls):
5         if cls._instance is None:
6             cls._instance = super().__new__(cls)
7             return cls._instance
8
9     def insert_feedback(self, user_id, message):
10        print("Data saved for " + user_id + ": " + message)
11
12 class FeedbackController:
13     def submit(self, user_id, occupancy):
14         db = SystemDatabase()
15         db.insert_feedback(user_id, "Occupancy: " + str(occupancy))
16
17 controller1 = FeedbackController()
18 controller1.submit("Student_1", 85)
19 controller1.submit("Student_2", 20)
20
21 db1 = SystemDatabase()
22 db2 = SystemDatabase()
23
24 print("Are both database objects the same?")
25 print(db1 is db2)
```

5. Class Diagram

